COLLEGE PRACTICAL EXAM & VIVA MASTER MANUAL

ALGO CRAFT: Data Structure Arena

Comprehensive Project Walkthrough, Core Algorithm Analysis & Top 25 Viva Questions

DOMAIN	CORE STRUCTURES	IMPLEMENTATION	CANDIDATE
Linear Data Structures	Stack & Queue (LIFO / FIFO)	Pure Vanilla JS + Tailwind	Jayesh

1. Executive Summary & System Architecture

ALGO CRAFT: Data Structure Arena is an interactive educational suite designed to bridge theoretical CS fundamentals with practical visualization. Built without external heavyweight frameworks, it delivers 60 FPS performance, pointer-level tracking, and physical real-world metaphors for both Stacks and Queues.

- **Landing Hub (#landing-hub):** Startup router allowing instant entry into either the Stack Master or Queue Master realms.
- **Top Navigation Switcher:** Synchronous controls (■ Hub, ■ Play Queue, ■ Play Stack) enabling fluid realm swapping while maintaining user state.
- **Web Audio Synthesizer:** Audio engine leveraging native browser Web Audio API (AudioContext, oscillators) to generate tones for push, pop, enqueue, dequeue, peek, and error events offline with zero MP3 dependencies.
- **Concept Guides:** Interactive multi-tab curriculum modal providing time complexities, boundary conditions, and C code snippets for every level.

2. Queue Master (FIFO Realm) — Module Breakdown

A **Queue** is a linear data structure governed by the **First-In, First-Out (FIFO)** principle: the first element inserted is always the first one to be removed. Insertions occur at the REAR (Enqueue) and deletions take place at the FRONT (Dequeue). Both core operations execute in **O(1) constant time**.

Level 1: The FIFO Factory (Conveyor Belt)

Concept: Physical FIFO shipping conveyor belt.
Pointers: Tracks FRONT (index 0) and REAR (index n-1).
FIFO Violation Trap: Clicking non-front packages triggers the 'FIFO Violation Modal', reinforcing to students that direct intermediate extraction is strictly illegal in Queue ADTs.

Level 2: Theme Park Cashier (Linear Array Queue & False Overflow)

Capacity: Fixed-size static array (MAX = 5).
Boundary Conditions:
- *Queue Overflow:* rear == MAX - 1
- *Queue Underflow:* front == -1 or front > rear

Crucial Viva Topic (False Overflow): When elements are dequeued, front moves forward, creating dead empty slots at the beginning. If rear == MAX - 1, no further enqueue is possible even though memory is vacant. This memory wastage problem necessitates Circular Queues.

Level 3: Traffic Control & Emergency Room (Queue Variants)

Variant 1 (Linear Queue): Vehicles lined up at a traffic light.
Variant 2 (Circular Queue): Solves False Overflow via modulo arithmetic:
- *Enqueue:* rear = (rear + 1) % MAX
- *Dequeue:* front = (front + 1) % MAX
- *Full Condition:* (rear + 1) % MAX == front
- *Empty Condition:* front == -1

Variant 3 (Priority Queue): Elements possess priority tags (High: ■ Ambulance, Med: ■ Bus, Low: ■ Sedan). Dequeue always extracts the highest-priority item first, overriding strict arrival time.

Level 4: Train Switchyard (Double-Ended Queue / Deque)

Concept: A generalized queue permitting insertion and deletion at **both ends**.
Operations: addFirst(), addLast(), removeFirst(), removeLast(), peekFirst(), peekLast().
CS Variants: *Input-Restricted Deque* (insert at one end, delete at both) and *Output-Restricted Deque* (delete at one end, insert at both).

Level 5: Queue Trivia Arcade

Interactive 5-question exam testing FIFO concepts, modulo math, error conditions, and complexities with live score tracking and heart lives.

3. Stack Master (LIFO Realm) — Module Breakdown

A **Stack** is a linear data structure adhering to the **Last-In, First-Out (LIFO)** principle: the most recently pushed element is the first one popped. All operations occur at a single pointer named TOP. Push, Pop, and Peek execute in **O(1) constant time**.

Level 1: The LIFO Cafeteria (Core Stack Fundamentals)

Analogy: Spring-loaded cafeteria plate dispenser.
Pointers: Single pointer TOP (starts at -1).
Boundary Conditions: Stack Overflow (top >= MAX - 1 in array mode MAX = 6); Stack Underflow (top == -1).
LIFO Violation: Clicking plates below top triggers violation modal.
Live C Code: Synchronously highlights C syntax for push(), pop(), and peek().

Level 2: Polish Notation & Postfix Lab

Sub-Lab A (Postfix Expression Evaluator): Left-to-right scan. Operands push to stack; operators pop B then A, calculate $A \text{ op } B$, and push result. Example: 2 3 1 * + 9 - yields **-4**.
Sub-Lab B (Infix to Postfix Converter): Shunting-Yard algorithm using operator stack with precedence (^ > * / > + -) and associativity.

Level 3: Syntax Sentinel (Balanced Parentheses & Brackets)

Validation: Validates matching pairs for (), {}, [].
Algorithm: Pushes opening brackets; pops on closing bracket and verifies type match.
Error Catching: Premature Underflow (e.g. ")(), Type Mismatch (e.g. "(]"), Unclosed Brackets (e.g. "(()"). Features a step-by-step Execution Trace Table.

Level 4: Monotonic Mountain (Next & Previous Greater Element)

Optimization: Replaces naive $O(N^2)$ brute force with an **O(N) linear time** Monotonic Decreasing Stack.
Operation: Scans right-to-left. Elements smaller than current are popped as they can never be the next greater for any element to the left. Includes radar height visualizer.

Level 5: Base & String Transformer

String Reversal: Pushing characters to stack and popping them naturally inverts order in $O(N)$ time.
Radix Converter: Repeated division of decimal N by target base B, pushing remainders (N % B) to stack, then popping from MSB to LSB.

Level 6: Dynamic Heap & Call Stack Recursion

Linked List Stack: Heap nodes [info | next*] allocated dynamically via malloc(); eliminates static capacity limits.
Call Stack Simulator: Visualizes execution of factorial(4). Shows activation records pushing on invocation ($n=4 \to 3 \to 2 \to 1$) and unwinding upon base case return.

Level 7: Stack Arcade Boss Quiz & Cheatsheet

8 comprehensive CS viva questions with score multipliers, streak tracking, heart lives, celebratory confetti, and integrated formula cheatsheet.

4. Top 25 Data Structure Viva Questions & Answers

These 25 questions cover the core concepts, boundary cases, mathematical proofs, and architectural details commonly probed during practical viva examinations:

Q1: What is the fundamental difference between a Stack and a Queue?

A Stack is a LIFO (Last-In, First-Out) structure where both insertion and deletion occur at the same end called TOP. A Queue is a FIFO (First-In, First-Out) structure where insertion occurs at the REAR and deletion occurs at the FRONT.

Q2: State the Overflow and Underflow boundary conditions for an array Stack.

Overflow occurs when attempting to push into a full stack (top >= MAX - 1 in a 0-indexed array). Underflow occurs when attempting to pop from an empty stack (top == -1).

Q3: What is False Overflow in a Linear Queue and how is it resolved?

In a linear queue, when elements are dequeued, front moves forward leaving vacant slots at the front. If rear == MAX - 1, enqueue fails even though memory is available. This False Overflow is solved by a Circular Queue wrapping pointers via modulo arithmetic.

Q4: State the Full and Empty conditions for a Circular Queue of capacity N.

Empty Condition: front == -1. Full Condition: (rear + 1) % N == front.

Q5: What are the time complexities of Push, Pop, Enqueue, and Dequeue?

All four operations operate in O(1) constant time because elements are directly accessed via pointers (top, front, rear) without shifting elements.

Q6: Why do compilers convert Infix expressions to Postfix expressions?

Infix expressions require operator precedence rules, associativity, and parentheses brackets. Postfix (Reverse Polish) notation removes all parentheses and ambiguity, allowing single-pass linear time O(N) evaluation using an operand stack.

Q7: Detail the algorithm for evaluating a Postfix expression using a Stack.

Scan tokens left to right. If operand, push to stack. If operator op, pop B then A, compute A op B, and push result back. Final result is on top of stack. (Crucial: operand order matters for division A/B and subtraction A-B).

Q8: How does bracket validation work using a Stack?

Push opening brackets '(', '{', '['. When closing bracket is read, check if stack is empty (premature underflow) or if top does not match closing type (mismatch). At end of string, stack must be empty.

Q9: What is a Monotonic Stack and how does it find Next Greater Element (NGE)?

A Monotonic Stack maintains elements in strictly increasing or decreasing order. For NGE, a monotonic decreasing stack scans right-to-left, popping elements smaller than current, reducing complexity from O(N^2) brute force to O(N) linear time.

Q10: What is a Double-Ended Queue (Deque)?

A Deque is a linear list allowing insertion and deletion from both ends: addFirst(), addLast(), removeFirst(), and removeLast().

Q11: Contrast Input-Restricted Deque vs. Output-Restricted Deque.

Input-restricted allows insertion at only one end, but deletion at both ends. Output-restricted allows deletion at only one end, but insertion at both ends.

Q12: How is a Priority Queue implemented efficiently?

Most efficiently implemented via a Binary Heap (Max-Heap / Min-Heap) providing O(log N) enqueue and O(log N) dequeue. Unsorted arrays require O(1) enqueue but O(N) dequeue.

Q13: Contrast Array implementation vs. Linked List implementation of a Stack.

Array stack: fixed size, potential overflow, zero pointer overhead, cache locality. Linked stack: dynamic growth (no overflow), but consumes extra memory for next* pointers.

Q14: Explain the role of the Call Stack in function recursion.

Each recursive invocation pushes an Activation Record (stack frame with parameters, local variables, return address). Upon reaching base condition, frames pop off (unwinding). Infinite recursion leads to Stack Overflow.

Q15: How to convert a Decimal integer to Binary using a Stack?

Repeatedly divide N by 2, pushing remainders (N % 2) onto the stack. Update N = N // 2 until N == 0. Pop all items: popped sequence yields binary digits from MSB to LSB.

Q16: How do you implement a Queue using two Stacks? What is the complexity?

Use Stack1 for enqueue (push). For dequeue: if Stack2 is empty, pop all elements from Stack1 into Stack2 (reversing order to FIFO), then pop from Stack2. Amortized time complexity is O(1).

Q17: How do you implement a Stack using two Queues?

For push(x): enqueue x to empty Q2, dequeue all elements from Q1 and enqueue into Q2, then swap Q1 and Q2. Pop is then an O(1) dequeue from Q1.

Q18: What is the Prefix equivalent of (A + B) * C?

Prefix: * + A B C. Postfix: A B + C *.

Q19: In a circular queue of size N, how many elements can be stored if empty slot is reserved?

At most N - 1 elements, reserving one slot so full condition ((rear + 1) % N == front) does not collide with empty condition (front == -1).

Q20: Differentiate peek() and pop().

peek() inspects the top element without removing it (top pointer unchanged). pop() returns the top element and removes it (decrements top).

Q21: Name 4 major real-world applications of Stacks.

1. Undo/Redo operations in editors (Ctrl+Z). 2. Browser history navigation (Back button). 3. Compiler syntax parsing & expression evaluation. 4. Function call stack execution.

Q22: Name 4 major real-world applications of Queues.

1. Operating system CPU process scheduling (Round Robin). 2. Printer print job spooling. 3. Breadth-First Search (BFS) graph traversal. 4. Async message buffers (Kafka, RabbitMQ).

Q23: Why is Web Audio API utilized instead of standard audio files?

Web Audio synthesizes frequencies mathematically using oscillators and gain envelopes in-browser. It eliminates network lag, 404 missing asset errors, requires zero bandwidth, and operates completely offline.

Q24: What is the space complexity comparison between Array Stack and Linked Stack?

Array Stack: O(MAX) pre-allocated memory. Linked Stack: O(K) where K is active elements, plus 8 bytes pointer overhead per node.

Q25: What key boundary scenarios can you demonstrate in this project?

1. Stack Overflow: push 6 plates, attempt 7th to show overflow alert. 2. LIFO / FIFO Violation: click lower plate or rear package to show violation alert. 3. Circular Queue: show rear wrapping around index 4 to 0. 4. Postfix Evaluation: step through '2 3 1 * + 9 -' yielding -4.

5. Rapid-Fire Reference Matrix

Property	Stack (LIFO)	Queue (FIFO)

Core Principle	Last-In, First-Out (LIFO)	First-In, First-Out (FIFO)
Insertion Point	TOP (Push)	REAR (Enqueue)
Deletion Point	TOP (Pop)	FRONT (Dequeue)
Pointers Required	1 pointer (top)	2 pointers (front, rear)
Initial Empty Pointers	top = -1	front = -1, rear = -1
Full Condition (Array)	top == MAX - 1	rear == MAX - 1 (Linear)
Empty Condition	top == -1	front == -1 or front > rear
Circular Queue Full	N/A	(rear + 1) % MAX == front
Time Complexity	O(1) Push / Pop / Peek	O(1) Enqueue / Dequeue
Graph Traversal	Depth-First Search (DFS)	Breadth-First Search (BFS)